Skenario Database & Aplikasi Feverr

1. Skenario Aplikasi

Bagian ini mendokumentasikan skenario-skenario penggunaan utama pada aplikasi Feverr, mencakup interaksi pengguna, alur proses, dan operasi database yang terlibat.

1.1 Skenario Registrasi dan Login Pengguna

Skenario: Registrasi dan Login Pengguna	
Aktor	Pengguna baru / pengguna terdaftar
Pemicu	Pengguna mengakses halaman /register atau /login
Pra-Kondisi	Pengguna belum memiliki sesi aktif
Langkah-langkah	1. Pengguna mengisi form (nama, username, email, password, role) 2. Sistem memvalidasi input dan mengecek keunikan email/username di tabel users 3. Password di-hash menggunakan Bcrypt (10 rounds) sebelum disimpan 4. Baris baru diinsert ke tabel users dengan balance = 0 5. Pada login berikutnya: NextAuth.js memverifikasi password hash 6. Jika global_user_id belum ada: sistem melakukan login/register ke JigsawCoin API 7. global_user_id disimpan ke kolom users.global_user_id 8. JWT session dibuat dengan payload: user_id, username, role, globalUserId
Pasca-Kondisi	Pengguna memiliki akun aktif dan sesi JWT terenkripsi. Akun JigsawCoin tersinkronisasi.
Pengecualian / Alternatif	1. Email/username sudah terdaftar: sistem mengembalikan error 400 2. JigsawCoin API tidak tersedia: login tetap berhasil, global_user_id diisi saat login berikutnya 3. Password salah: sistem mengembalikan error autentikasi

Fitur auto-sync JigsawCoin menggunakan logika berikut di lib/auth.ts:

1. Coba login ke JigsawCoin API dengan kredensial yang sama
2. Jika gagal (status 401), coba register akun baru di JigsawCoin
3. Jika sudah terdaftar (status 409), lakukan lookup berdasarkan email
4. Simpan global_user_id ke database Feverr

1.2 Skenario Pencarian dan Browsing Gig

Skenario: Pencarian Gig dengan Redis Caching	
Aktor	Buyer / pengguna tidak login
Pemicu	Pengguna mengakses /search dengan atau tanpa parameter (q, category, maxPrice, rating)
Pra-Kondisi	Halaman search dapat diakses publik
Langkah-langkah	1. Server membaca parameter query dari URL (q, category, maxPrice, minRating) 2. Server membentuk cache key: 'gigs:all' atau 'gigs:category:{kategori}' 3. Server mencoba membaca data dari Redis (redis.get(cacheKey)) 4. Cache HIT: data dikembalikan langsung dari Redis (<20ms) 5. Cache MISS: server menjalankan SQL JOIN ke PostgreSQL, lalu menyimpan hasilnya ke Redis (TTL: 3600 detik) 6. Filter in-memory diterapkan: pencarian teks (q), batas harga (maxPrice), rating minimum (minRating) 7. Hasil gig yang sudah difilter ditampilkan dalam grid kartu 8. Filter diperbarui real-time melalui URL params (debounce 300ms)
Pasca-Kondisi	Pengguna melihat daftar gig yang sesuai filter. Sistem mencatat hit/miss cache.
Pengecualian / Alternatif	1. Redis tidak tersedia: sistem otomatis fallback ke PostgreSQL (try-catch handler), tanpa downtime 2. Tidak ada gig yang cocok: halaman menampilkan pesan kosong 3. Koneksi database gagal: error 500 dikembalikan

1.2.1 Alur Redis Caching

Tahap	Deskripsi
Permintaan Masuk	User membuka halaman /search (dengan/tanpa filter kategori)
Cek Cache Redis	Server memeriksa key "gigs:all" atau "gigs:category:{kategori}" di Redis
Cache HIT	Data dikembalikan langsung dari Redis (<20 ms). Filter tambahan (q, maxPrice, rating) diterapkan di server secara in-memory
Cache MISS	Server menjalankan query JOIN ke PostgreSQL (gigs JOIN users), menyimpan hasilnya ke Redis dengan TTL 1 jam

Invalidasi Cache	Cache dihapus (redis.del) saat seller membuat, mengedit, atau menghapus gig (route POST/PUT/DELETE /api/gigs)
Failsafe Fallback	Jika Redis tidak tersedia (error koneksi), sistem langsung membaca dari PostgreSQL tanpa downtime bagi pengguna

Tabel 4.1 - Alur Kerja Redis Caching pada Pencarian Gig

Perbandingan performa sebelum dan sesudah implementasi Redis:

- Tanpa Redis: Query SQL JOIN langsung ke PostgreSQL, latensi 600ms - 1.500ms
- Dengan Redis (Cache HIT): Latensi < 20ms (penurunan hingga 98%)
- Overhead database berkurang hingga 80% untuk query pencarian redundan

1.3 Skenario Pembuatan dan Manajemen Gig oleh Seller

Skenario: Pembuatan Gig Baru	
Aktor	Seller (pengguna dengan role seller atau both)
Pemicu	Seller mengakses /seller/gigs/new dan mengisi form gig
Pra-Kondisi	Pengguna telah login sebagai seller/both
Langkah-langkah	1. Seller mengisi form: judul, kategori, deskripsi, harga, hari pengiriman, tag, gambar 2. Seller mengunggah gambar portfolio ke Cloudinary melalui /api/upload 3. Cloudinary mengembalikan secure URL yang disimpan di state form 4. Seller submit form: POST /api/gigs dipanggil dengan data lengkap 5. Server memvalidasi sesi dan role pengguna 6. Data gig diinsert ke tabel gigs dengan seller_id dari session 7. Cache Redis 'gigs:all' dan 'gigs:category:{kategori}' dihapus (cache invalidation) 8. Gig baru langsung muncul di halaman search
Pasca-Kondisi	Gig baru tersimpan di database. Cache Redis di-invalidasi. Gig visible di pencarian.
Pengecualian / Alternatif	1. Pengguna bukan seller: akses ditolak (redirect ke login) 2. Validasi form gagal: error ditampilkan per field 3. Upload gambar gagal: error Cloudinary ditampilkan, form masih bisa disimpan

1.4 Skenario Pemesanan Jasa (Checkout)

Skenario: Pembuatan Pesanan Baru	
Aktor	Buyer
Pemicu	Buyer menekan tombol 'Pesan Sekarang' pada halaman detail gig
Pra-Kondisi	Buyer telah login; buyer bukan seller dari gig tersebut; saldo Rupiah buyer >= harga gig
Langkah-langkah	1. Buyer mengisi form instruksi dan menekan 'Konfirmasi Pesanan' 2. Server memvalidasi sesi dan membaca data gig (harga, seller_id) 3. Server membaca saldo buyer dari tabel users 4. Validasi saldo: jika balance < total_price, kembalikan error 400 5. INSERT ke tabel orders: buyer_id, gig_id, seller_id, status='pending', total_price, buyer_instructions 6. UPDATE users SET balance = balance - total_price (deduct saldo buyer) 7. INSERT ke tabel transactions: type='debit', amount=total_price, description='Order for {gig.title}' 8. Buyer diarahkan ke halaman dashboard pesanan
Pasca-Kondisi	Order berstatus 'pending' tersimpan. Saldo buyer berkurang. Transaksi debit tercatat di ledger.
Pengecualian / Alternatif	1. Saldo tidak cukup: error 400 'Insufficient balance', buyer diarahkan ke halaman dompet 2. Gig tidak ditemukan: error 404 3. Buyer mencoba memesan gig miliknya sendiri: perlu validasi tambahan (belum diimplementasi)

1.5 Skenario Pengelolaan Status Pesanan

Pesanan di Feverr mengikuti alur status yang ketat dengan otorisasi berbeda di setiap tahap.

Status	Aktor Pemicu	Kondisi & Aksi Database
pending	Sistem (saat order dibuat)	Saldo buyer dikurangi (balance - total_price); transaksi debit dicatat di tabel transactions; order tersimpan
in_progress	Seller	Status diperbarui; seller mulai mengerjakan pesanan; chat aktif
delivered	Seller	Status diperbarui; delivery_files[] diisi dengan nama file yang dikirim seller

completed	Buyer (via review)	Buyer mengirim review; saldo seller ditambah (balance + total_price); transaksi credit seller dicatat; average_rating gig diperbarui
cancelled	Sistem / Admin	Status diperbarui (skenario saat ini belum diimplementasi penuh)

Tabel 4.2 - Alur Status Pesanan dan Operasi Database

Skenario: Penyelesaian Pesanan dan Pembayaran Seller	
Aktor	Buyer (mengirim review)
Pemicu	Buyer menekan tombol 'Kirim Ulasan & Selesaikan' pada pesanan berstatus delivered
Pra-Kondisi	Order berstatus 'delivered'; buyer belum mengirim review untuk order ini
Langkah-langkah	1. Buyer mengisi rating (1-5 bintang) dan komentar 2. POST /api/reviews: review diinsert ke tabel reviews 3. Status order diupdate menjadi 'completed' 4. Saldo seller ditambah: UPDATE users SET balance = balance + total_price 5. Transaksi credit seller dicatat di tabel transactions 6. Rata-rata rating gig diperbarui: UPDATE gigs SET average_rating = AVG(rating), review_count = COUNT(*) 7. Cache Redis untuk kategori gig di-invalidasi
Pasca-Kondisi	Order selesai. Seller menerima pembayaran. Rating gig diperbarui. Ledger transaksi tercatat.
Pengecualian / Alternatif	1. Review sudah ada untuk order ini: sistem menampilkan konfirmasi sudah direview 2. Order belum berstatus delivered: tombol review tidak ditampilkan

1.6 Skenario Chat dalam Pesanan

Skenario: Pertukaran Pesan dalam Pesanan	
Aktor	Buyer dan Seller
Pemicu	Salah satu pihak mengetik dan mengirim pesan di ChatBox pada halaman detail order
Pra-Kondisi	Order ada dan pengguna adalah buyer atau seller dari order tersebut

Langkah-langkah	1. Pengguna mengetik pesan dan menekan 'Kirim' 2. POST /api/messages/{orderId}: pesan diinsert ke tabel messages 3. Server memvalidasi bahwa pengirim adalah buyer atau seller dari order 4. Pesan tersimpan dengan sender_id, order_id, dan content 5. Halaman di-refresh (loadData dipanggil ulang) untuk menampilkan pesan terbaru 6. Pesan ditampilkan dengan gaya bubble berbeda untuk pengirim vs penerima
Pasca-Kondisi	Pesan tersimpan di database. Kedua pihak dapat melihat riwayat percakapan.
Pengecualian / Alternatif	1. Pengguna bukan bagian dari order: akses ditolak (error 403) 2. Pesan kosong: validasi frontend dan backend mencegah pengiriman

2. Skenario Transaksi Keuangan

Sistem keuangan Feverr mengintegrasikan dua mata uang: Rupiah (IDR) yang disimpan di PostgreSQL lokal, dan JigsawCoin (JGC) yang dikelola oleh layanan eksternal JigsawCoin API. Penukaran antara keduanya difasilitasi melalui server-side proxy untuk menghindari masalah CORS dan melindungi API key.

2.1 Arsitektur Server-Side Proxy

Mengapa Server-Side Proxy Diperlukan?

1. MASALAH CORS: Browser secara default memblokir request langsung dari JavaScript
   frontend ke domain JigsawCoin API yang berbeda.

2. KEAMANAN API KEY: JIGSAWCOIN_API key harus disimpan di environment variable
   server (.env). Memanggil API dari frontend akan membocorkan key ke publik.

Solusi: Semua komunikasi ke JigsawCoin API dilakukan server-side melalui Next.js API Routes (/api/wallet/info dan /api/wallet/swap).

2.2 Alur Swap Bidirectional IDR ↔ JGC

Aspek	JGC → IDR (Swap Masuk)	IDR → JGC (Swap Keluar)
Input	Jumlah JGC yang ingin ditukar	Jumlah Rupiah yang ingin ditukar (min. Rp10, kelipatan Rp10)
Validasi saldo	Cek saldo JGC via JigsawCoin API (server-side)	Cek balance di PostgreSQL (kolom users.balance)
Langkah 1	Potong JGC via JigsawCoin API (amount negatif)	Potong Rupiah di PostgreSQL (UPDATE users SET balance - amount)
Langkah 2	Tambah Rupiah di PostgreSQL (UPDATE users SET balance + amount)	Tambah JGC via JigsawCoin API (amount positif)
Langkah 3	Catat transaksi credit di tabel transactions	Catat transaksi debit di tabel transactions
Proteksi rollback	Jika potong JGC gagal, proses berhenti (Rupiah tidak ditambah)	Jika tambah JGC gagal, proses berhenti (Rupiah tidak dikurangi)
Kurs	1 JGC = Rp1.000 (tetap)	1 JGC = Rp1.000 (tetap); presisi 0.01 JGC

Tabel 5.1 - Perbandingan Alur Swap JGC ↔ IDR

2.3 Skenario Swap JGC ke IDR (Top Up Rupiah)

Skenario: Swap JGC → IDR (Tukar Koin ke Rupiah)	
Aktor	Pengguna yang memiliki saldo JigsawCoin
Pemicu	Pengguna memasukkan jumlah JGC dan menekan 'Tukar ke Rupiah' di halaman /settings/wallet
Pra-Kondisi	Pengguna telah login; akun JigsawCoin tersinkronisasi (global_user_id ada)
Langkah-langkah	1. Pengguna memilih arah swap 'JGC → IDR' dan memasukkan jumlah JGC 2. Frontend menampilkan estimasi hasil (jumlah JGC x Rp1.000) 3. POST /api/wallet/swap: request dikirim dengan { amount, direction: 'jgc_to_idr' } 4. Server mengecek saldo JGC via JigsawCoin API (GET /wallet/balance/{globalUserId}) 5. Validasi: jika saldo JGC tidak cukup, kembalikan error 400 6. Server mengirim request deduct ke JigsawCoin API (POST /wallet/transaction, amount negatif) 7. Jika deduct berhasil: UPDATE users SET balance = balance + rupiahAmount 8. INSERT ke tabel transactions: type='credit', description='Swap from X JGC' 9. Frontend menampilkan notifikasi sukses dan memuat ulang saldo
Pasca-Kondisi	Saldo JGC berkurang di JigsawCoin. Saldo Rupiah bertambah di PostgreSQL. Transaksi credit tercatat.
Pengecualian / Alternatif	1. Saldo JGC tidak cukup: error 400 'Insufficient JigsawCoin balance' 2. JigsawCoin API gagal saat deduct: proses berhenti, Rupiah tidak ditambah (atomic protection) 3. global_user_id tidak ada: error 400 'JigsawCoin account not linked'

2.4 Skenario Swap IDR ke JGC (Konversi Rupiah ke Koin)

Skenario: Swap IDR → JGC (Tukar Rupiah ke Koin)	
Aktor	Pengguna yang memiliki saldo Rupiah di Feverr

Pemicu	Pengguna memasukkan jumlah Rupiah (min. Rp10, kelipatan Rp10) dan menekan 'Tukar ke JigsawCoin'
Pra-Kondisi	Pengguna telah login; saldo Rupiah mencukupi; global_user_id tersedia
Langkah-langkah	1. Pengguna memilih arah swap 'IDR → JGC' dan memasukkan jumlah Rupiah 2. Frontend menghitung estimasi JGC yang diterima (jumlah Rupiah / 1000, 2 desimal) 3. POST /api/wallet/swap: request dikirim dengan { amount, direction: 'idr_to_jgc' } 4. Server membaca saldo Rupiah dari PostgreSQL (SELECT balance FROM users) 5. Validasi: jika balance < rupiahToSwap, kembalikan error 400 6. Server mengirim request tambah JGC ke JigsawCoin API (amount positif) 7. Jika tambah JGC berhasil: UPDATE users SET balance = balance - rupiahToSwap 8. INSERT ke tabel transactions: type='debit', description='Swap to X JGC'
Pasca-Kondisi	Saldo Rupiah berkurang. Saldo JGC bertambah di JigsawCoin. Transaksi debit tercatat di ledger.
Pengecualian / Alternatif	1. Saldo Rupiah tidak cukup: error 400 'Insufficient Rupiah balance' 2. JigsawCoin API gagal saat tambah koin: proses berhenti, Rupiah tidak dikurangi 3. Jumlah tidak sesuai kelipatan Rp10: validasi frontend mencegah pengiriman

2.5 Kurs Konversi dan Presisi Desimal

Feverr menggunakan kurs tetap (fixed rate) sebesar 1 JGC = Rp1.000. Presisi transaksi didukung hingga 0,01 JGC (setara Rp10), yang merupakan denominasi terkecil yang valid. Batasan kelipatan Rp10 pada swap IDR ke JGC memastikan hasil konversi selalu bulat di angka dua desimal, mencegah pembulatan curang atau kerugian nilai.

3. Skenario Manajemen Konten Seller

3.1 Skenario Upload Portofolio

Skenario: Upload Gambar Portofolio Gig	
Aktor	Seller
Pemicu	Seller menekan tombol '+' pada form gig untuk menambah gambar portofolio
Pra-Kondisi	Seller sedang mengisi atau mengedit form gig
Langkah-langkah	1. Seller memilih file gambar dari perangkat 2. Frontend mengirim POST /api/upload dengan FormData berisi file 3. Server membaca file sebagai ArrayBuffer, lalu mengonversi ke Buffer 4. Buffer dikirim ke Cloudinary menggunakan upload_stream ke folder 'feverr_portfolio' 5. Cloudinary mengembalikan secure_url gambar yang sudah dihosting di CDN 6. URL disimpan ke state form (portfolioImages[]) di frontend 7. Saat gig disimpan: array URL diinsert ke kolom portfolio_images[] di tabel gigs
Pasca-Kondisi	Gambar tersimpan di Cloudinary CDN. URL tersimpan di database. Server utama tidak menyimpan file.
Pengecualian / Alternatif	1. File bukan gambar atau terlalu besar: error dari Cloudinary 2. API Cloudinary tidak tersedia: error 500 dari /api/upload

3.2 Skenario Edit dan Nonaktifkan Gig

Skenario: Edit Gig dan Invalidasi Cache	
Aktor	Seller
Pemicu	Seller mengklik 'Edit' pada daftar gig-nya dan menyimpan perubahan
Pra-Kondisi	Seller telah login; gig milik seller yang bersangkutan
Langkah-langkah	1. Seller membuka halaman edit gig, data diambil dari GET /api/gigs/{id} 2. Seller mengubah data (judul, harga, deskripsi, status aktif, dsb.) 3. PUT /api/gigs/{id}: server memvalidasi ownership (seller_id == session.user.id) 4. Data gig diupdate di PostgreSQL

	5. Cache Redis di-invalidasi: redis.del('gigs:all') dan redis.del('gigs:category:{kategori}') 6. Perubahan langsung terlihat di halaman search karena cache sudah diperbarui
Pasca-Kondisi	Gig diperbarui di database. Cache Redis di-invalidasi. Perubahan langsung visible.
Pengecualian / Alternatif	1. Seller mencoba edit gig milik orang lain: error 403 Forbidden 2. Gig tidak ditemukan: error 404 3. Redis tidak tersedia saat invalidasi: operasi diabaikan, cache akan expire setelah TTL